Tema:

Actividades de Programación en Ambiente Cliente/Servidor Unidad 2

Docente:

Jenner Noel Che Méndez

Carrera:

Ingeniería en informática

Alumnos:

José Abimael Cisneros Escamilla

Uriel Barrales Martínez

Semestre:

7AI

Fecha:

2 de noviembre de 2026

Índice

Act 1. Modelar requisitos antes de programar 3
Act 2. Contrato OpenAPI y revisión cruzada 9
Act 3. Implementación del backend (API + ORM + BD) 13
Act 4. Construcción de un cliente web independiente 17
Act 5. Pruebas sistemáticas, defectos y regresión 19

Act 1. Modelar requisitos antes de programar

Descripción del problema

El Instituto Tecnológico necesita un sistema que permita controlar el préstamo de laptops y proyectores. Los estudiantes y docentes podrán solicitar equipos que se encuentren disponibles. Cada equipo contará con un identificador, tipo, marca, modelo, número de inventario y estado.

Cuando una persona solicite un equipo, se deberá registrar la fecha del préstamo y la fecha esperada de devolución. Por reglas de control interno, una persona no podrá tener más de dos préstamos activos al mismo tiempo, y un equipo que ya esté prestado no podrá prestarse nuevamente hasta ser liberado. Cuando el equipo sea devuelto, se registrará la fecha real de devolución. Finalmente, los administradores serán los únicos facultados para registrar nuevos equipos y modificar su estado operativo.

Actores del sistema

Estudiante: Consulta el catálogo de equipos y solicita préstamos.

Docente: Consulta el catálogo de equipos y solicita préstamos.

Administrador: Registra nuevos equipos en el inventario y modifica su estado.

Sistema de préstamos: Verifica disponibilidad de equipos, estado del usuario y cumplimiento de reglas de negocio.

Requisitos Funcionales

- RF-01: Permitir la consulta de todos los equipos registrados y de un equipo específico por su identificador.
- RF-02: Permitir el filtrado de equipos según su disponibilidad o estado actual.
- RF-03: Permitir a los administradores registrar nuevos equipos, actualizar sus datos/estado y eliminarlos si no están prestados.
- RF-04: Permitir el registro de un préstamo vinculando a un usuario activo con un equipo disponible y estableciendo una fecha esperada de devolución.
- RF-05: Cambiar automáticamente el estado del equipo a "prestado" al concretarse un préstamo.
- RF-06: Permitir la consulta de préstamos registrados (general e individual por ID).
- RF-07: Permitir el registro de la devolución de un equipo, registrando la fecha real de devolución, cambiando el préstamo a "devuelto" y liberando el equipo a "disponible".

Recursos Identificados

Concepto analizado	¿Es recurso REST?	Justificación de diseño
Usuario (usuarios)	Sí	Tiene identidad propia (id), datos propios y realiza préstamos (generaliza a estudiante y docente).
Equipo (equipos)	Sí	Tiene identidad (id), número de inventario y ciclo de vida propio (disponible, prestado, mantenimiento).
Préstamo (prestamos)	Sí	Representa una transacción con identidad propia, fechas de control y estado que vincula a Usuario y Equipo.
Administrador	No	No requiere un endpoint aislado; es un rol o tipo dentro del recurso Usuario.
Fecha / Estado / Marca / Modelo	No	Carecen de identidad independiente; son únicamente atributos descriptivos dentro de Equipo o Préstamo.

Atributos principales de cada recurso

A. Recurso: Usuario (/api/v1/usuarios)

- Atributos: id (entero), nombre (cadena), correo (cadena), tipo (cadena: estudiante/docente/admin), activo (booleano).
- Representación JSON preliminar:

JSON

```
{
  "id": 25,
  "nombre": "Carlos Pérez",
  "correo": "carlos@itchina.edu.mx",
  "tipo": "estudiante",
  "activo": true
}
```

B. Recurso: Equipo (/api/v1/equipos)

- Atributos: id (entero), tipo (laptop | proyector), marca (cadena), modelo (cadena), numeroInventario (cadena única), estado (disponible | prestado | mantenimiento).
- Representación JSON preliminar:

JSON

```
{
  "id": 8,
  "tipo": "laptop",
  "marca": "Dell",
```

```
  "modelo": "Latitude 5540",
  "numeroInventario": "ITCH-PC-008",
  "estado": "disponible"
}
```

C. Recurso: Préstamo (/api/v1/prestamos)

- Atributos: id (entero), usuarioId (entero), equipoId (entero), fechaPrestamo (fecha), fechaEsperadaDevolucion (fecha), fechaDevolucion (fecha o null), estado (activo | devuelto).
- Representación JSON preliminar:

JSON

```
{
  "id": 152,
  "usuarioId": 25,
  "equipoId": 8,
  "fechaPrestamo": "2026-10-06",
  "fechaEsperadaDevolucion": "2026-10-08",
  "fechaDevolucion": null,
  "estado": "activo"
}
```

Relaciones entre recursos

El modelo relaciona tres entidades principales donde Préstamo actúa como entidad asociativa:

- Usuario (1) → (N) Préstamo: Un usuario puede realizar históricamente muchos préstamos, pero cada préstamo pertenece a un solo usuario (usuarioId).
- Equipo (1) → (N) Préstamo: Un equipo puede aparecer históricamente en muchos préstamos a lo largo del tiempo, pero cada registro de préstamo se refiere a un equipo específico (equipoId).
- Restricción no estructural (Regla de Negocio): Aunque la relación permite N préstamos históricos por equipo, solamente puede existir un préstamo en estado "activo" por equipo al mismo tiempo.

Operaciones y endpoints propuestos.

Recurso	Operación	Método HTTP	URI Propuesta (Base: /api/v1)
Usuarios	Listar usuarios	GET	/usuarios
Usuarios	Registrar usuario	POST	/usuarios
Usuarios	Modificar usuario	PATCH	/usuarios/{id}
Usuarios	Eliminar usuario	DELETE	/usuarios/{id}
Equipos	Listar equipos	GET	/equipos
Equipos	Consultar equipo	GET	/equipos/{id}
Equipos	Registrar equipo	POST	/equipos
Equipos	Modificar equipo	PATCH	/equipos/{id}

Equipos	Eliminar equipo	DELETE	/equipos/{id}
Préstamos	Listar préstamos	GET	/prestamos
Préstamos	Consultar préstamo	GET	/prestamos/{id}
Préstamos	Crear préstamo	POST	/prestamos
Préstamos	Registrar devolución	PATCH	/prestamos/{id}

Reglas de negocio

- RN-01 (Existencia y estado activo del usuario): El usuario solicitante debe existir en el sistema y tener su cuenta activa. SI usuario.activo == false ENTONCES rechazar préstamo.
- RN-02 (Límite máximo de préstamos): Un usuario no podrá tener más de dos préstamos activos simultáneamente. SI prestamosActivos >= 2 ENTONCES rechazar nuevo préstamo.
- RN-03 (Existencia y disponibilidad del equipo): El equipo solicitado debe existir y encontrarse disponible. SI equipo.estado != "disponible" ENTONCES rechazar préstamo.
- RN-04 (Cambio automático de estado al prestar): Todo equipo recién creado inicia como "disponible", y cuando se registra un préstamo válido, su estado cambia automáticamente a "prestado" (equipo.estado = "prestado").
- RN-05 (Sincronización en devolución): Solo puede devolverse un préstamo cuyo estado sea "activo". Al registrar la devolución: prestamo.estado = "devuelto", se guarda fechaDevolucion, y el equipo vuelve a estar disponible (equipo.estado = "disponible").
- RN-06 (Permisos administrativos y protección de borrado): Solo un usuario con rol administrador puede ejecutar POST /equipos, PATCH /equipos/{id} y DELETE /equipos/{id}. Además, no se puede eliminar un equipo que se encuentre actualmente "prestado".

Validaciones

- correo es obligatorio y tiene formato email: Verifica estructura sintáctica antes de tocar la lógica del sistema.
- usuarioId y equipoId deben ser números enteros positivos (> 0): Comprueba el tipo de dato y rango numérico válido.
- fechaEsperadaDevolucion es obligatoria, tiene formato YYYY-MM-DD y es futura: Asegura que el campo exista, sea una fecha ISO válida y coherente.
- tipo de equipo solo acepta "laptop" o "proyector": Restringe el valor de entrada a un conjunto permitido (enum).

- El usuario tiene máximo dos préstamos activos: Depende del estado actual almacenado en la base de datos.
- El equipo solicitado debe estar "disponible": Evalúa la disponibilidad real del recurso en el inventario.
- Solamente el administrador registra o modifica equipos: Evalúa permisos y roles dentro de la organización.

Posibles errores y códigos HTTP.

Situación / Escenario	Código HTTP	Descripción del comportamiento
Consulta o actualización exitosa	200 OK	La petición se procesó correctamente y devuelve el recurso o lista.
Recurso creado (Equipo o Préstamo)	201 Created	El préstamo o equipo fue validado y persistido exitosamente.
Equipo eliminado correctamente	204 No Content	El recurso fue eliminado sin contenido de retorno.
JSON mal formado, tipos incorrectos, falta fecha o ID negativo	400 Bad Request / 422 Unprocessable Content	La petición falla en la capa de validación de entrada antes de ejecutar reglas de negocio.
Cliente intenta operar sin iniciar sesión	401 Unauthorized	El usuario no ha proporcionado credenciales válidas.
Estudiante o docente intenta registrar/modificar un equipo	403 Forbidden	El usuario está autenticado pero no tiene rol de administrador (viola RN-06).
El usuarioId, equipoId o prestamoId no existe en la BD	404 Not Found	El recurso solicitado o referenciado no existe.
El usuario está suspendido / inactivo	409 Conflict	El JSON es válido, pero viola la regla RN-01.
El usuario ya tiene 2 préstamos activos	409 Conflict	Conflicto con la regla de negocio RN-02.
El equipo ya está prestado o en mantenimiento	409 Conflict	Conflicto con la regla de negocio RN-03.
Se intenta devolver un préstamo que ya fue devuelto antes	409 Conflict	Conflicto con la regla RN-05; evita dobles devoluciones.

Ejemplo de una petición y una respuesta

Petición HTTP

```
POST /api/v1/prestamos HTTP/1.1
Host: localhost:8000
Content-Type: application/json

{
  "usuarioId": 25,
  "equipoId": 8,
  "fechaEsperadaDevolucion": "2026-10-18"
}
```

Respuesta Exitosa (Response 201 Created)

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 152,
  "usuarioId": 25,
  "equipoId": 8,
  "fechaPrestamo": "2026-10-16",
  "fechaEsperadaDevolucion": "2026-10-18",
  "fechaDevolucion": null,
  "estado": "activo"
}
```

Respuesta de Error de Negocio (Response 409 Conflict)

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "error": {
    "codigo": "EQUIPO_NO_DISPONIBLE",
    "mensaje": "El equipo solicitado se encuentra actualmente prestado."
  }
}
```

Act 2. Contrato OpenAPI y revisión cruzada

Evidencia 1: resumen de requisitos y reglas de negocio

Para construir el contrato formal bajo el enfoque **API-first**, partimos de los tres recursos principales (Usuario, Equipo, Préstamo) y las reglas de negocio definidas.

- RN-01: El usuario solicitante debe existir en el sistema.
- RN-02: El usuario debe encontrarse en estado "activo".
- RN-03: El usuario puede tener como máximo dos préstamos activos simultáneamente.
- RN-04: El equipo solicitado debe existir en el inventario.
- RN-05: El equipo debe encontrarse en estado "disponible".
- RN-06: Al crearse el préstamo, el estado del equipo cambia automáticamente a "prestado".
- RN-07: Al registrarse la devolución, el préstamo cambia a "devuelto" y el equipo regresa a "disponible".

Evidencia 2: Archivo openapi-v0.1.yaml inicial

Este es el primer borrador (v0.1.0) entregado por el equipo proveedor al equipo consumidor. Es sintácticamente válido en yaml, pero contiene omisiones y ambigüedades típicas de un primer diseño que serán detectadas en la revisión cruzada.

https://github.com/Utze147/ComparteLab/blob/main/Unidad2/docs/openapi-v0.1.yaml

Evidencia 3: Bitácora de Revisión Cruzada

Cuestionario de inspección del consumidor:

Pregunta del consumidor	¿El borrador lo explica claramente?	Diagnóstico Técnico
¿Cuál es la URL base?	**Sí**	Indica http://localhost:8000/api/v1.
¿Cómo consulto equipos y un equipo por ID?	**Parcial**	Define las rutas, pero equipoId no restringe 0 o negativos.
¿Cómo filtro equipos disponibles?	**No**	El parámetro estado es string libre; no especifica si es "disponible", "libre" o "DISPONIBLE".
¿Qué JSON debo enviar en POST /prestamos y qué campos son obligatorios?	**No**	NuevoPrestamo no tiene bloque required, no tiene examples ni restringe propiedades adicionales.
¿Qué formato usan las fechas?	**No**	Declara type: string sin format: date (¿08/10/2026 o 2026-10-08?).
¿Qué devuelve un préstamo creado (201)?	**No**	La respuesta 201 no define el schema de salida.
¿Qué ocurre si el equipo no existe o está ocupado?	**No**	No están documentadas las respuestas 400, 404 ni 409, ni existe un esquema uniforme de Error.

Tabla de observaciones técnicas y respuesta del proveedor

ID	Elemento	Problema encontrado por el consumidor	Severidad	Propuesta del Consumidor	Resolución del Proveedor
OBS-01	POST /prestamos y Schemas	Ningún esquema define qué campos son obligatorios ni incluye ejemplos JSON.	Alta	Agregar listas required, additionalProperties: false en entradas y bloques examples.	ACEPTADA. Se agregan required y ejemplos en todos los esquemas y operaciones.
OBS-02	Equipo.estado, Equipo.tipo y Prestamo.estado	Como consumidor no puedo determinar qué valores acepta estado o tipo porque el contrato solo indica string.	Media	Agregar enum	ACEPTADA. Se restringen todos los estados y tipos con enum.
OBS-03	Manejo de Errores (400, 404, 409, 422)	No existe un formato uniforme de error ni se documentan los conflictos de negocio	Alta	Crear el esquema Error (codigo, mensaje, detalles) y reutilizar	ACEPTADA. Se incorpora el esquema Error dentro de un objeto contenedor y ejemplos para 409 Conflict.

				respuestas 400/404/409.	
OBS-04	Campos de Fecha	No se indica el formato de fechaEsperadaDevolucion ni fechaDevolucion, y fechaDevolucion no permite null cuando el préstamo sigue activo.	Media	Utilizar format: date (ISO 8601 YYYY-MM-DD) y permitir ["string", "null"] en fechaDevolucion.	ACEPTADA. Se aplica format: date y tipo nullable en OpenAPI 3.1.
OBS-05	IDs en Path y Body (equipoId, usuarioId, prestamoId)	Los identificadores aceptan 0 o números negativos, y los parámetros de ruta están repetidos sin reutilización.	Media	Añadir minimum: 1 y extraer EquipoId y PrestamoId a #/components/parameters/.	ACEPTADA. Se parametrizan con $ref y minimum: 1.
OBS-06	Operaciones de Equipos	Faltan en el contrato las operaciones administrativas para crear (POST), modificar (PATCH) y eliminar (DELETE) equipos.	Alta	Incorporar POST /equipos, PATCH /equipos/{equipoId} y DELETE /equipos/{equipoId}.	ACEPTADA CON MODIFICACIÓN. Se agregan las rutas creando esquemas específicos (EquipoCreate y EquipoUpdate) para que el cliente no envíe id ni estado al crear.
OBS-07	Consulta por usuario	El consumidor propone crear el endpoint GET /prestamosUsuario/{id} para ver los préstamos de una persona.	Baja	Crear ruta /prestamosUsuario/{usuarioId}.	RECHAZADA. prestamosUsuario introduce un nombre acoplado/acción innecesario; en diseño RESTful se resuelve mediante query params (GET /prestamos?usuarioId=25&estado=activo) o sub-recurso /usuarios/{id}/prestamos.

Evidencia 4: archivo openapi.yaml corregido y aprobado

Contrato definitivo en OpenAPI 3.1.2 tras aplicar el linter y todas las observaciones aceptadas.

https://github.com/Utze147/ComparteLab/blob/main/Unidad2/docs/openapi.yaml

Evidencia 5: Defensa técnica de las decisiones tomadas

Separación entre openapi: 3.1.2 y version: 1.0.0: Se especificó 3.1.2 para indicar la versión del estándar OpenAPI utilizado (lo cual nos permite usar arreglos de tipos como ["string", "null"] nativos de JSON Schema moderno en fecha_devolucion), mientras que 1.0.0 representa la primera versión aprobada de nuestro contrato de negocio tras superar los borradores v0.1 y v0.2.

Eliminación de ambigüedades mediante enum, minimum y format: En la revisión cruzada se comprobó que declarar un campo como string o integer obliga al consumidor a adivinar. Se incorporaron listas cerradas (enum) para estados (disponible, prestado, mantenimiento y activo, devuelto), minimum: 1 para todos los identificadores y format: date para fechas, garantizando que el cliente construya peticiones válidas sin consultar el código fuente del servidor.

Separación de esquemas de lectura y escritura (Equipo vs. EquipoCreate): En lugar de usar el mismo esquema Equipo en el POST /equipos, se diseñó EquipoCreate omitiendo id y estado. Esto protege la regla de negocio donde el servidor asigna el ID y garantiza que todo equipo nuevo comience como "disponible".

Distinción semántica entre 422 Unprocessable Content y 409 Conflict: Se acordó que cualquier error de estructura, tipos incorrectos, campos faltantes o longitudes inválidas se reporte con 422 (DATOS_INVALIDOS), mientras que un JSON sintácticamente perfecto que viole una regla de negocio (usuario inactivo, límite de 2 préstamos alcanzado, equipo ya prestado o devolución duplicada) devuelva 409 Conflict.

Contrato único de errores (ErrorResponse): Todos los fallos de la API (404, 409, 422) devuelven exactamente la estructura {"error": {"codigo": "...", "mensaje": "...", "detalles": [...]}}. Esto permite que el equipo consumidor programe una sola función manejadora de errores en el frontend.

Rechazo de URIs orientadas a procedimientos (OBS-07): Se rechazó la propuesta de crear /prestamosUsuario/{id} para preservar los principios RESTful aprendidos en la Actividad 1, manteniendo sustantivos puros (/prestamos) y delegando el filtrado por usuario a parámetros de consulta (?usuario_id=25).

Act 3. Implementación del backend (API + ORM + BD)

Arquitectura y estructura del proyecto por capas

El proyecto se implementa en Python utilizando FastAPI, SQLAlchemy ORM y PostgreSQL. Para evitar mezclar responsabilidades, aplicamos la separación por capas con la siguiente estructura de archivos:

```
api-prestamos/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── errors.py
│   ├── services/
│   │   ├── __init__.py
│   │   ├── equipos.py
│   │   └── prestamos.py
│   └── routers/
│       ├── __init__.py
│       ├── equipos.py
│       └── prestamos.py
└── requirements.txt
```

Modelo relacional y mapeo orm

El dominio relaciona tres entidades principales: Usuario (1:N) → Préstamo (N:1) ← Equipo, donde Prestamo vincula quién solicita con qué equipo, manteniendo el historial de transacciones sin permitir el borrado físico (DELETE) para proteger la integridad histórica.

- Archivo database.py: Configura el motor de conexión a PostgreSQL y la sesión por petición (SessionLocal) que agrupa el trabajo del ORM en una unidad de interacción.
- Archivo models.py: Define las tablas, restricciones estructurales (unique=True, ForeignKey) y las relaciones bidireccionales mediante relationship().

Esquemas de validación y serialización

Separamos el modelo ORM del contrato JSON de la API mediante Pydantic. Al crear un equipo, el cliente no envía el campo estado porque el servidor impone la regla de que todo equipo nuevo inicie como "disponible".

Manejo uniforme de errores

Para que cualquier error (404, 409, 422) devuelva siempre la misma estructura JSON ({"error": {"codigo": "...", "mensaje": "...", "detalles": ...}}), definimos nuestra excepción de dominio AppError.

Capa de servicios: reglas de negocio, consultas seguras y transacciones

En esta capa implementamos las consultas parametrizadas por ORM, el control de unicidad sin exponer errores internos de PostgreSQL y las transacciones TODO o NADA.

Capa de routers y aplicación principal

Registra la creación inicial de tablas (create_all), monta los routers bajo /api/v1 e intercepta tanto AppError como RequestValidationError para garantizar el formato uniforme de errores.

Memoria técnica: seguridad sql, transacciones y pruebas de éxito/fallo

Consultas parametrizadas vs. concatenación sql

- Práctica Inyección SQL: Construir consultas concatenando entradas del usuario ("SELECT * FROM equipos WHERE estado = '" + estado + "'") permite alterar la sentencia SQL.
- Implementación segura aplicada: Se utiliza el ORM de SQLAlchemy (select(Equipo).where(Equipo.estado == estado)) o sentencias parametrizadas con text("SELECT * FROM equipos WHERE estado = :estado"), separando los datos del código SQL.

Evidencia de Transacciones e Integridad

En POST /api/v1/prestamos y PATCH /api/v1/prestamos/{id} se modifican dos tablas relacionadas dentro de una misma unidad transaccional.

Al crear un préstamo: se ejecuta el INSERT en prestamos (estado = "activo") y el UPDATE en equipos (estado = "prestado") confirmándose juntos mediante un único db.commit(). Si ocurre una excepción intermedia, el bloque except ejecuta db.rollback(), impidiendo que exista un préstamo activo cuyo equipo siga figurando como "disponible".

Evidencia de ejecución: casos exitosos y casos que fallan correctamente

Casos Exitosos (200, 201, 204)

Prueba	Petición (Request)	Respuesta HTTP y JSON Obtenido (Response)
Crear equipo válido	POST /api/v1/equipos {"tipo": "laptop", "marca": "Dell", "modelo": "Latitude 5540", "numero_inventario": "ITCH-PC-008"}	201 Created {"id": 8, "tipo": "laptop", "marca": "Dell", "modelo": "Latitude 5540", "numero_inventario": "ITCH-PC-008", "estado": "disponible"}

Filtrar, buscar y paginar	GET /api/v1/equipos?estado=disponible&buscar=dell&limit=20&offset=0	200 OK [{"id": 8, "tipo": "laptop", "marca": "Dell", "modelo": "Latitude 5540", "numero_inventario": "ITCH-PC-008", "estado": "disponible"}]
Crear préstamo válido	POST /api/v1/prestamos {"usuario_id": 25, "equipo_id": 8, "fecha_esperada_devolucion": "2026-10-25"}	201 Created {"id": 152, "usuario_id": 25, "equipo_id": 8, "fecha_prestamo": "2026-10-16T19:00:00Z", "fecha_esperada_devolucion": "2026-10-25", "fecha_devolucion": null, "estado": "activo"} (Equipo 8 pasa a "prestado")
Registrar devolución	PATCH /api/v1/prestamos/152 {"fecha_devolucion": "2026-10-18"}	200 OK {"id": 152, ..., "fecha_devolucion": "2026-10-18", "estado": "devuelto"} (Equipo 8 vuelve a "disponible")

Casos que deben fallar correctamente (404, 409, 422)

Escenario de Prueba Fallida	**Petición (Request)**	**Código HTTP**	**JSON de Error Uniforme Obtenido**
Consultar recurso inexistente	GET /api/v1/equipos/9999	404 Not Found	{"error": {"codigo": "EQUIPO_NO_ENCONTRADO", "mensaje": "El equipo solicitado no existe."}}
Crear préstamo con usuario inexistente	POST /api/v1/prestamos (usuario_id: 9999)	404 Not Found	{"error": {"codigo": "USUARIO_NO_ENCONTRADO", "mensaje": "El usuario solicitado no existe."}}
Crear préstamo con equipo inexistente	POST /api/v1/prestamos (equipo_id: 9999)	404 Not Found	{"error": {"codigo": "EQUIPO_NO_ENCONTRADO", "mensaje": "El equipo solicitado no existe."}}
Crear préstamo con usuario inactivo	POST /api/v1/prestamos (usuario_id: 3)	409 Conflict	{"error": {"codigo": "USUARIO_INACTIVO", "mensaje": "El usuario no puede realizar préstamos."}}
Crear préstamo con equipo prestado	POST /api/v1/prestamos (equipo_id: 8 ocupado)	409 Conflict	{"error": {"codigo": "EQUIPO_NO_DISPONIBLE", "mensaje": "El equipo solicitado no está disponible."}}

Crear tercer préstamo activo	POST /api/v1/prestamos (usuario ya tiene 2 activos)	409 Conflict	{"error": {"codigo": "LIMITE_PRESTAMOS", "mensaje": "El usuario ya tiene dos préstamos activos."}}
Eliminar equipo prestado	DELETE /api/v1/equipos/8	409 Conflict	{"error": {"codigo": "EQUIPO_CON_PRESTAMO", "mensaje": "No se puede eliminar un equipo prestado."}}
Enviar ID negativo	POST /api/v1/prestamos (usuario_id: -5)	422 Unprocessable Content	{"error": {"codigo": "DATOS_INVALIDOS", "mensaje": "La solicitud contiene datos inválidos.", "detalles": [...]}}
Enviar fecha pasada	POST /api/v1/prestamos (fecha_esperada_devolucion: "2020-01-01")	422 Unprocessable Content	{"error": {"codigo": "DATOS_INVALIDOS", "mensaje": "La solicitud contiene datos inválidos.", "detalles": [{"msg": "Value error, La fecha de devolución debe ser futura."}]}}
Enviar estado no permitido	PATCH /api/v1/equipos/8 ({"estado": "roto"})	422 Unprocessable Content	{"error": {"codigo": "DATOS_INVALIDOS", "mensaje": "La solicitud contiene datos inválidos.", "detalles": [...]}}

Act 4. Construcción de un cliente web independiente

Arquitectura de Interoperabilidad

Para demostrar que el cliente y el servidor están completamente desacoplados tecnológicamente, la aplicación web se desarrolla en una carpeta totalmente separada que no comparte código con el backend en Python. La interoperabilidad se logra única y exclusivamente mediante HTTP, JSON y el contrato OpenAPI.

```
cliente-web/
├── index.html
└── app.js
```

- Ejecución independiente de procesos: Servidor Backend (Python + FastAPI): Se ejecuta en el puerto 8000.
- Cliente Frontend (HTML5 + JS): Se ejecuta en el puerto 5500 mediante un servidor estático.

Configuración de CORS en el Servidor

Dado que el navegador distingue orígenes por protocolo, host y puerto, al comunicarse desde http://localhost:5500 hacia http://localhost:8000 se aplica la política de seguridad CORS. Para permitirlo, actualizamos nuestro servidor FastAPI en app/main.py.

Demostración técnica y análisis de devtools

Al abrir las herramientas de desarrollo del y realizar acciones en la interfaz web, el alumno puede comprobar la anatomía real de las peticiones:

A. Solicitud GET /api/v1/equipos

- Request URL: http://localhost:8000/api/v1/equipos
- Request Method: GET
- Status Code: 200 OK
- Response Body (JSON recibido y deserializado en objeto JS):


```
[
  {
    "id": 8,
    "tipo": "laptop",
    "marca": "Dell",
    "modelo": "Latitude 5540",
    "numero_inventario": "ITCH-PC-008",
    "estado": "disponible"
  }
]
```

B. Solicitud POST /api/v1/equipos (Registrar)

- Request URL: http://localhost:8000/api/v1/equipos
- Request Method: POST
- Status Code: 201 Created
- Request Headers: Content-Type: application/json
- Payload (JSON enviado):

```
{
  "tipo": "laptop",
  "marca": "Lenovo",
  "modelo": "ThinkPad",
  "numero_inventario": "ITCH-PC-050"
}
```

- Response (JSON recibido):

```
{
  "id": 50,
  "tipo": "laptop",
  "marca": "Lenovo",
  "modelo": "ThinkPad",
  "numero_inventario": "ITCH-PC-050",
  "estado": "disponible"
}
```

Clasificación de errores y diagnóstico del cliente

El cliente diferencia claramente tres clases de problemas operativos:

1. Errores del Cliente (Frontend): Fallos de sintaxis en JavaScript, formularios mal enlazados o propiedades mal escritas en el DOM.
2. Errores HTTP / Reglas de Negocio: Respuestas del servidor con códigos 404, 409 o 422 (por ejemplo, intentar duplicar un número de inventario o registrar un préstamo con un equipo ocupado). El servidor respondió correctamente, pero la operación fue rechazada por el dominio, y la interfaz muestra el mensaje descriptivo en la alerta roja.
3. Errores de Red / CORS / Servidor Caído: Si se apaga FastAPI (uvicorn), la función fetch() lanza una excepción de red o el navegador bloquea por CORS. El cliente captura el fallo y despliega un aviso de conectividad sin colgar la aplicación.

Act 5. Pruebas sistemáticas, defectos y regresión

Preparación del ambiente de pruebas y datos controlados

Para garantizar que las pruebas sean aisladas, controladas y repetibles sin afectar información real, se configuró la base de datos de pruebas prestamos_test en PostgreSQL. Antes de ejecutar la matriz, se inicializó el entorno con los siguientes datos conocidos:

Tabla usuarios (Estado inicial):

ID	Nombre	Estado
1	Ana López	activo
2	Carlos Pérez	activo
3	Luis Gómez	inactivo

Tabla equipos (Estado inicial):

ID	Equipo (Marca / Modelo)	Número de Inventario	Estado
1	Dell Latitude	ITCH-PC-001	disponible
2	Epson PowerLite	ITCH-PRO-002	disponible
3	Lenovo ThinkPad	ITCH-PC-003	mantenimiento

Matriz de pruebas y resultados de ejecución

Se diseñaron previamente los casos positivos (CP), negativos (CN) y de integridad (CI) definiendo entrada, acción y resultado esperado antes de ejecutar las peticiones:

ID	Tipo	Caso de Prueba	Entrada / Acción	Resultado Esperado	Obtenido	Estado
CP-01	Positiva	Listar equipos	GET /api/v1/equipos	200 OK + lista JSON con estructura válida	200 OK	PASS
CP-02	Positiva	Crear equipo válido	POST /api/v1/equipos (JSON HP ProBook 440)	201 Created + estado: "disponible"	201 Created	PASS
CP-03	Positiva	Crear préstamo válido	POST /api/v1/prestamos (usuario_id: 1,	201 Created + préstamo activo	201 Created	PASS

			equipo_id: 1)			
CP-04	Positiva	Registrar devolución	PATCH /api/v1/prestamos/25 (fecha_devolucion)	200 OK + préstamo devuelto	200 OK	PASS
CN-01	Negativa	Consultar equipo inexistente	GET /api/v1/equipos/9999	404 Not Found (EQUIPO_NO_ENCONTRADO)	404 Not Found	PASS
CN-02	Negativa	Datos inválidos / ID negativo	POST /api/v1/equipos (tipo: "automovil", marca: "") y GET /equipos/-5	422 Unprocessable Content (DATOS_INVALIDOS)	422 Unprocessable	PASS
CN-03	Negativa	Equipo ocupado	POST /api/v1/prestamos sobre equipo_id: 1 ya prestado	409 Conflict (EQUIPO_NO_DISPONIBLE)	409 Conflict	PASS
CN-04	Negativa	Usuario inactivo	POST /api/v1/prestamos con usuario_id: 3 (inactivo)	409 Conflict (USUARIO_INACTIVO)	201 Created (1.ª ejec.) / 409 (Retest)	FAIL → PASS
CN-05	Negativa	Tercer préstamo activo	POST /api/v1/prestamos cuando usuario_id: 1 ya tiene 2 activos	409 Conflict (LIMITE_PRESTAMOS)	409 Conflict	PASS
CN-06	Negativa	Devolución duplicada	Segundo PATCH /api/v1/prestamos/25	409 Conflict (PRESTAMO_YA_DEVUELTO)	409 Conflict	PASS

CI-01	Integridad	Inventario duplicado (UNIQUE)	POST /api/v1/equipos con numero_inventario: "ITCH-PC-001"	No duplicar en BD; responder 409 (INVENTARIO_DUPLICADO) sin exponer error SQL	409 Conflict	PASS
CI-02	Integridad	Clave foránea de usuario (FK)	POST /api/v1/prestamos con usuario_id: 99999	No crear relación inválida; rechazar con 404	404 Not Found	PASS
CI-03	Integridad	Cambio de estado al prestar	GET /api/v1/equipos/1 tras crear préstamo válido	equipo.estado cambia a "prestado" y prestamo.estado es "activo"	"prestado"	PASS
CI-04	Integridad	Liberación de equipo al devolver	GET /api/v1/equipos/1 tras PATCH /prestamos/25	prestamo.estado = "devuelto" y equipo.estado = "disponible"	"disponible"	PASS
CI-05	Integridad	Fallo en transacción (Rollback)	Error provocado entre INSERT préstamo y UPDATE equipo	Rollback completo: sin préstamo parcial y equipo sigue "disponible"	Rollback exitoso	PASS

Código fuente: https://github.com/Utze147/ComparteLab/tree/main/Unidad2/api-prestamos

Evidencia detallada de requests, responses y estado en base de datos

1. Prueba positiva y de integridad en creación

- Solicitud (POST /api/v1/prestamos):

```
{
  "usuario_id": 1,
  "equipo_id": 1,
  "fecha_esperada_devolucion": "2026-10-25"
}
```

- Respuesta obtenida (201 created):

```
{
  "id": 25,
  "usuario_id": 1,
  "equipo_id": 1,
  "fecha_prestamo": "2026-10-19T15:00:00Z",
  "fecha_esperada_devolucion": "2026-10-25",
  "fecha_devolucion": null,
  "estado": "activo"
}
```

- Comprobación posterior en API y base de datos:
  - Antes: equipo.estado = 'disponible'.
  - Después (SELECT id, estado FROM equipos WHERE id = 1;): id = 1 | estado = 'prestado'.

2. Pruebas negativas: validación vs. conflicto de negocio

- Caso CN-02 (Datos inválidos → 422 Unprocessable Content):
  - Request (POST /api/v1/equipos): {"tipo": "automovil", "marca": "", "modelo": "X", "numero_inventario": "1"}
  - Response (422):

```
{
  "error": {
    "codigo": "DATOS_INVALIDOS",
    "mensaje": "La solicitud contiene datos inválidos.",
    "detalles": [
      {"loc": ["body", "tipo"], "msg": "Input should be 'laptop' or 'proyector'"},
      {"loc": ["body", "marca"], "msg": "String should have at least 2 characters"},
      {"loc": ["body", "numero_inventario"], "msg": "String should have at least 3 characters"}
    ]
  }
}
```

- Caso CN-03 (Regla de negocio: Equipo ocupado → 409 Conflict):
  - Request (POST /api/v1/prestamos): {"usuario_id": 2, "equipo_id": 1, "fecha_esperada_devolucion": "2026-10-26"}.
  - Response (409): {"error": {"codigo": "EQUIPO_NO_DISPONIBLE", "mensaje": "El equipo solicitado no está disponible."}}

- Caso CN-05 (Límite de 2 préstamos activos + Verificación en BD):
    - Precondición: El usuario 1 ya tiene 2 préstamos activos (#25 y #26).
    - Request (POST /api/v1/prestamos): Intento de crear el préstamo activo #3.
    - Response (409): {"error": {"codigo": "LIMITE_PRESTAMOS", "mensaje": "El usuario ya tiene dos préstamos activos."}}.
    - Evidencia de integridad (GET /api/v1/prestamos?usuario_id=1&estado=activo y SQL SELECT * FROM prestamos WHERE usuario_id = 1;): Continúa devolviendo exactamente 2 registros, demostrando que el rechazo 409 no insertó datos accidentales.

3. Prueba de transacciones y rollback

Para demostrar que el INSERT del préstamo y el UPDATE del equipo ocurren en una sola transacción atómica, se introdujo temporalmente una excepción controlada en el servicio:

```
db.add(prestamo)
db.flush()
raise Exception("Fallo provocado para prueba")
equipo.estado = "prestado"
db.commit()
```

Evidencia en PostgreSQL antes y después del fallo:

- Consulta SELECT * FROM prestamos WHERE usuario_id = 2;: 0 filas (el db.rollback() revirtió el INSERT tras el flush).
- Consulta SELECT id, estado FROM equipos WHERE id = 2;: id: 2, estado: 'disponible' (el equipo nunca quedó bloqueado ni inconsistente).

Bitácora de defectos, investigación de causa raíz, corrección y regresión

Durante la primera ejecución del caso CN-04 (Usuario inactivo), se detectó un defecto de severidad Alta (viola una regla de negocio importante) que fue documentado, corregido y verificado siguiendo el ciclo profesional de pruebas:

1. Registro en bitácora de defectos

Campo	**Valor Registrado**
ID del Defecto	DEF-004
Fecha	19/10/2026
Caso relacionado	CN-04 (Probar usuario inactivo)
Endpoint	POST /api/v1/prestamos
Descripción	El sistema permite registrar un préstamo a un usuario cuyo estado es inactivo.

Entrada utilizada	{"usuario_id": 3, "equipo_id": 2, "fecha_esperada_devolucion": "2026-10-26"} (Usuario 3 tiene activo = false).
Resultado Esperado	409 Conflict con código "USUARIO_INACTIVO".
Resultado Obtenido	201 Created (creó el préstamo indebidamente).
Severidad	**Alta** (Viola la regla de negocio RN-02).
Estado inicial	Abierto

2. Investigación de causa raíz y corrección aplicada

Causa raíz identificada: Al inspeccionar app/services/prestamos.py, se observó que el servicio consultaba la existencia del usuario (usuario = db.get(Usuario, datos.usuario_id)), pero omitía evaluar la propiedad booleana usuario.activo antes de continuar con el préstamo. Código de corrección aplicado en services/prestamos.py.

3. Cierre del defecto, retest y prueba de regresión

Campo de Cierre	**Valor**
Defecto	DEF-004
Corrección	Validación de if not usuario.activo lanzando AppError(409, "USUARIO_INACTIVO", ...)
Archivo modificado	app/services/prestamos.py
Resultado / Retest	Corregido / Retest Aprobado (PASS con 409 Conflict)
Estado final	Cerrado (19/10/2026)

Ejecución de regresión: Para demostrar que la validación de usuario inactivo no rompió la funcionalidad de usuarios activos ni otras reglas, se volvieron a ejecutar:

- CP-03 (Préstamo válido con usuario activo 1) → 201 Created (PASS).
- CN-04 (Préstamo con usuario inactivo 3) → 409 Conflict (PASS).
- CN-03 (Préstamo con equipo ocupado 1) → 409 Conflict (PASS).
- CI-03 (Cambio de estado del equipo a "prestado") →PASS.

Automatización de pruebas con pytest y testclient

Para transformar las comprobaciones manuales en evidencia automática y repetible, se instalaron pytest y httpx (pip install pytest httpx) y se implementó la suite de pruebas en la carpeta tests/ utilizando TestClient de FastAPI.

https://github.com/Utze147/ComparteLab/tree/main/Unidad2/api-prestamos/tests

Análisis de reto avanzado: condición de carrera por concurrencia

Si dos clientes (Cliente A y Cliente B) envían simultáneamente POST /api/v1/prestamos solicitando el mismo equipo_id: 1, ambos procesos podrían ejecutar la lectura SELECT al mismo tiempo y observar equipo.estado == "disponible" antes de que cualquiera de los dos confirme su COMMIT, generando dos préstamos activos para un solo equipo.

Para proteger esta regla crítica más allá de la validación en Python, en entornos de producción se debe acompañar la transacción con bloqueo pesimista de fila en la base de datos (SELECT ... FOR UPDATE mediante .with_for_update() en SQLAlchemy) o mediante un índice único parcial en PostgreSQL (CREATE UNIQUE INDEX idx_equipo_prestamo_activo ON prestamos (equipo_id) WHERE estado = 'activo';).